TEMA 3 – Estrategias básicas de búsqueda

Introducción

Estrategias sobre una Representación por Espacio de Estados

- Búsqueda no informada o a ciegas
  - Introducción
  - Búsqueda sobre árboles
  - Estrategias
  - Evitar estados repetidos – Búsqueda sobre grafos
- Búsqueda heurística
  - Introducción
  - Estrategias

Estrategias sobre una Representación por Reducción

- Búsqueda no informada o a ciegas
  - Introducción
  - Búsqueda sobre árboles
  - Estrategias
- Búsqueda heurística
  - Introducción
  - Características de las funciones de evaluación en grafos YO
  - Estrategia YO*

Funciones heurísticas

- El efecto de la precisión heurística en el rendimiento
- Diseñando funciones heurísticas admisibles

INTRODUCCIÓN

Para resolver un problema **hay que:**

- Establecer **Objetivo**: estados que satisfacen los criterios para ser meta o subproblemas establecidos como terminales (primitivos).
- Aplicar un **Método de Búsqueda**: proceso para examinar y encontrar qué secuencia de acciones permite obtener un estado objetivo o dar como resuelto el problema. Para ello, el método debe decidir:
  - qué acciones y
  - qué estados/subproblemas considerar.

Un método de búsqueda utiliza como entrada un problema y devuelve como salida una secuencia de acciones que conducen a un objetivo final.

Introducción

En general, un diseño simple de este proceso debe "formular, buscar, ejecutar"

función RESOLVER-PROBLEMAS(percepción) **devuelve** una acción

entradas: percepción, una percepción
estático: problema: una formulación del problema
sec: una secuencia de acciones
estado o sub-prob actual
objetivo o problema terminal (primitivo)

Repetir
problema ← FORMULAR-PROBLEMA()
Inicio ← FORMULAR-INICIO()
objetivo ← FORMULAR-OBJETIVO()
sec ← BÚSQUEDA(problema)
acción ← PRIMERO(secuencia)
devolver acción
estado/sub-prob ← ACTUALIZAR(estado/sub-prob,percepción)

Presupone

- Entorno estático ➔ porque la formulación y búsqueda del problema se hace sin prestar atención a cualquier cambio que puede ocurrir en el entorno.
- Se conoce el estado inicial o problema.
- Entorno discreto ➔ enumerar "las líneas de acción alternativas".
- Entorno determinista ➔ Las soluciones son secuencias de acciones y se ejecutan sin prestar atención a las percepciones.

Estudiaremos:

1. El proceso de formulación del problema.
2. Métodos para la función BÚSQUEDA.

Introducción

Independientemente de la representación utilizada, la estructura subyacente será un árbol o grafo ordinario o YO descritos por los nodos y los arcos.

La representación de un nodo "n": estructura con estas 5 componentes mínimo:

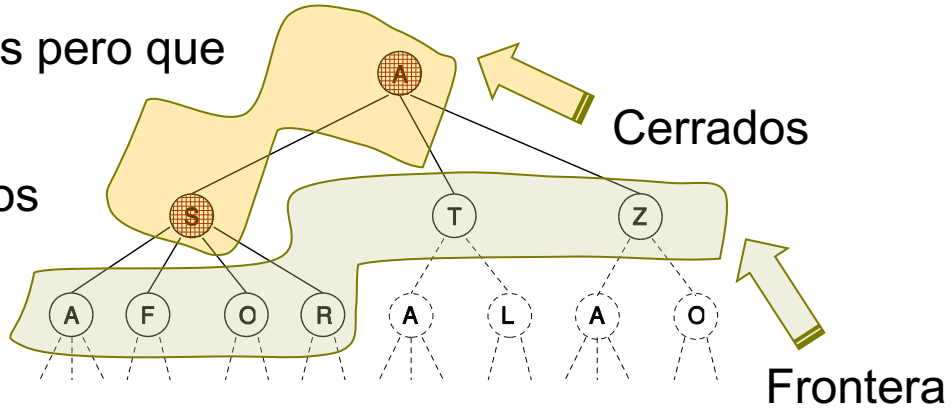
- ESTADO o SUBPROBLEMA: identifica al nodo.
- NODO PADRE: nodo en el árbol/grafo de búsqueda que ha generado este nodo.
- ACCIÓN: la acción/operador "a" que se aplicará al padre para generar el nodo.
- COSTO DEL CAMINO: el costo "g(n)" de un camino indicado por los punteros a los padres.
- PROFUNDIDAD: el número de pasos a los largo del camino.

A partir de esta estructura del nodo se va construyendo el árbol/grafo de búsqueda.

Introducción

Frontera: Colección de nodos generados pero que aún no han sido expandidos.

Cerrados: Colección de nodos generados y expandidos.



La estrategia de búsqueda es una función que selecciona de la frontera el siguiente nodo a expandir.

Asumiremos que la colección de nodos se implementa como una cola con operaciones como las siguientes:

VACIA(cola), devuelve verdadera si la cola está vacía.

BORRAR(cola,criterio), borra de la cola los nodos que verifican el criterio.

BORRAR-PRIMERO(cola), devuelve el primer elemento y lo borra de la cola.

INSERTA(elemento,cola), inserta elemento y devuelve cola.

INSERTAR-TODO(elementos,cola), inserta elementos y devuelve cola.

Introducción

La abstracción en la formulación de problemas

La formulación de los problemas se están estableciendo en términos de estado inicial o problema, función sucesor, test objetivo o subproblemas terminales (primitivos) y costo. Esta formulación es razonable aunque se pueden omitir muchos aspectos.

- De los estados o subproblemas: aspectos climáticos, presión de los brazos de robots, etc.
- De las acciones: tiempo requerido, gasto de combustible, contaminación, etc.

El proceso de eliminar detalles de una representación se le llama **abstracción**.

Introducción

Midiendo el rendimiento de la resolución del problema

La salida de un método de resolución de problemas es fallo o una solución.

Se evalúa el rendimiento de un método por medio de la:

- **Completitud**: ¿Está garantizado que se encuentre una solución, cuando exista?
- **Optimalidad:** ¿Se encuentra la solución óptima?
- **Complejidad en tiempo:** ¿Cuánto tarda en encontrar una solución?
- **Complejidad en espacio:** ¿Cuánta memoria se necesita para la búsqueda?

La complejidad en tiempo y espacio siempre se consideran con respecto a alguna medida de la dificultad del problema. Como en general tendremos el árbol de búsqueda representado de forma implícita por el estado inicial y la función sucesor, la complejidad la expresaremos en términos de tres cantidades:

- *b*, factor de ramificación o el máximo número de sucesores de cualquier nodo;
- *d*, profundidad del nodo objetivo más superficial;
- *m*, longitud máxima de cualquier camino en el espacio de búsqueda.

ESTRATEGIAS SOBRE UNA REPRESENTACIÓN POR ESPACIO DE ESTADOS

Realizamos el estudio de esta sección siguiendo el siguiente esquema:

- Métodos de búsqueda
  - sin información o a ciegas
    - Anchura
    - Costo uniforme
    - Profundidad
  - con información o heuristicos
    - Primero el mejor avara
    - A
    - BPMR

Estrategias sobre una repres. por espacio de estados

BÚSQUEDA NO INFORMADA O A CIEGAS

- **Introducción**

Impracticable para problemas complejos pero forman el esqueleto de todas las demás. No se hace uso de información específica del problema o del dominio concreto (la información sobre el beneficio o utilidad de pasar de un estado a otro no se considera). En esta situación se adopta algún criterio arbitrario aunque fijo.

Un **problema** de búsqueda consta de 4 **componentes**:

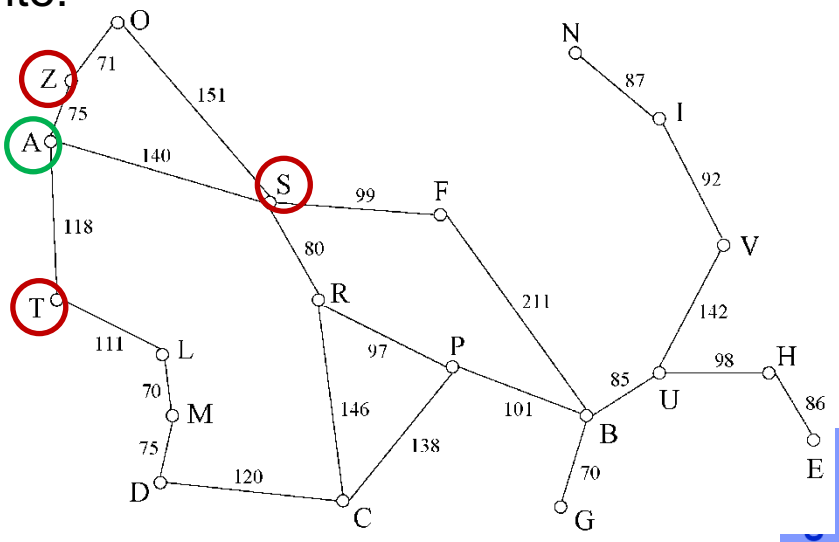
1. El **estado inicial** en el que comienza el agente.
2. Descripción de las posibles **acciones**.

Formulación más común:

SUCESOR(x) devuelve conjunto de pares ordenados <acción, sucesor>

acción ➔ una de las acciones legales en x

sucesor ➔ estado alcanzable desde x

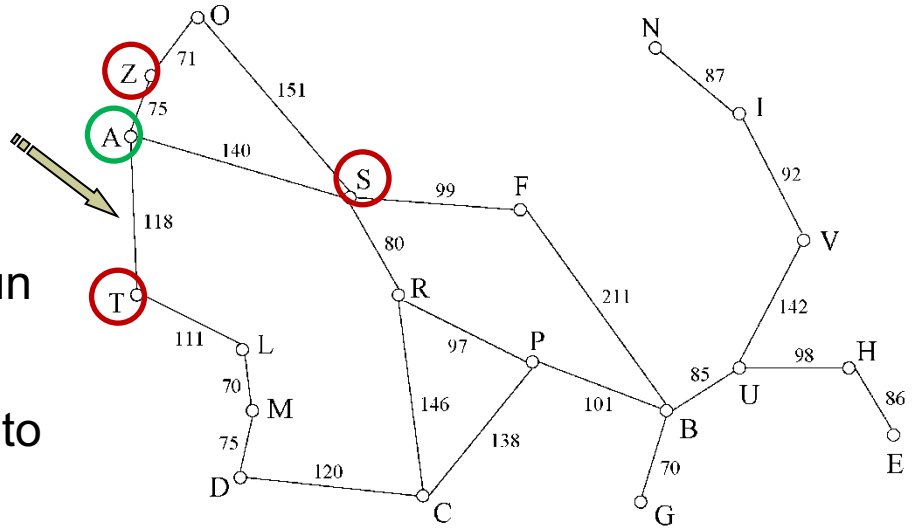


Estrategias sobre una repres. por espacio de estados

Búsqueda no informada o a ciegas

Implícitamente el estado inicial y la función sucesor definen el espacio de estados - conjunto de todos los estados alcanzables.

Camino: estados conectados por una secuencia de acciones



3. El **test objetivo**, el cual determina si un estado es un estado objetivo.
4. Una **función costo** que asigna un costo numérico a cada camino

Costo individual = costo no negativo de una acción "*a*" desde "*x*" a "*y*"
c(x,a,y) o abreviadamente c(x,y)

Costo del camino = suma de los costos individuales

La calidad de la solución se mide por la función costo del camino.

Una **solución óptima** tiene el costo más pequeño entre todas las soluciones.

Estrategias sobre una repres. por espacio de estados

Búsqueda no informada o a ciegas

En general, en la búsqueda de las soluciones se realiza lo siguiente:

Continuando con el ejemplo de la ruta:

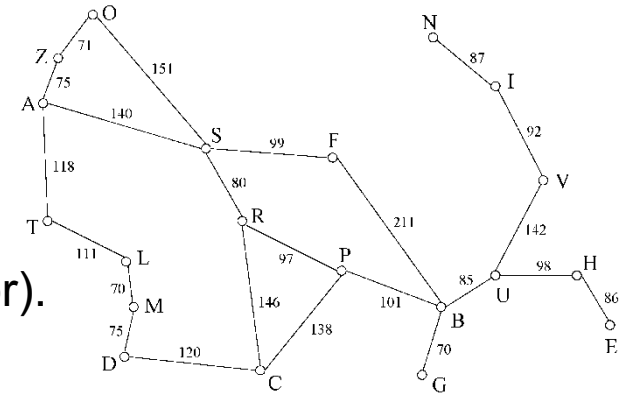
Nodo raíz: corresponde al estado inicial, En(A).

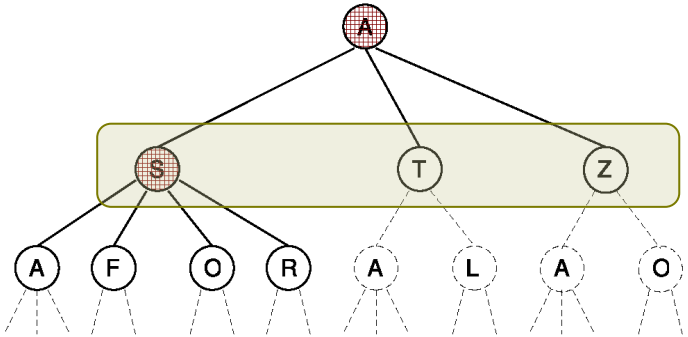
Comprobar si éste es un estado objetivo.

Expandir el estado actual (aplicamos la función sucesor).

3 nuevos estados: En(S), En(T) y En(Z)

Escoger cuál de estas tres posibilidades consideramos. Seguir…….





El estado a expandir está determinado por la estrategia de búsqueda.

IMPORTANTE: distinguir entre Espacio de Estados y el Árbol de Búsqueda.

En el ejemplo: 20 estados -- número infinito de nodos.

Estrategias sobre una repres. por espacio de estados

Búsqueda no informada o a ciegas

- **Búsqueda sobre árboles**

función BÚSQUEDA-ÁRBOLES(problema,frontera) **devuelve** una solución o fallo

```
frontera ← INSERTA(HACER-NODO(ESTADO-INICIAL[problema]),frontera)
bucle hacer
   si VACIA(frontera) entonces devolver fallo
   nodo ← BORRAR-PRIMERO(frontera)
   si TEST-OBJETIVO[problema] aplicado al ESTADO[nodo] es cierto entonces devolver SOLUCION(nodo)
   frontera ← INSERTAR-TODO(EXPANDIR(nodo,problema),frontera)
```

---

función EXPANDIR(nodo,problema) **devuelve** un conjunto de nodos

```
sucesores ← conjunto vacío
para cada (acción,resultado) en SUCESOR[problema](ESTADO[nodo]) hacer
   s ← un nuevo NODO
   ESTADO[s] ← resultado
   NODO-PADRE[s] ← nodo
   ACCIÓN[s] ← acción
   COSTO-CAMINO[s] ← COSTO-CAMINO[nodo]+COSTO-INDIVIDUAL(nodo,acción,s)
   PROFUNDIDAD[s] ← PROFUNDIDAD[nodo]+1
   añadir s a sucesores
devolver sucesores
```

Distintas estrategias ordenan la frontera de distinta forma

Estrategias sobre una repres. por espacio de estados

Búsqueda no informada o a ciegas

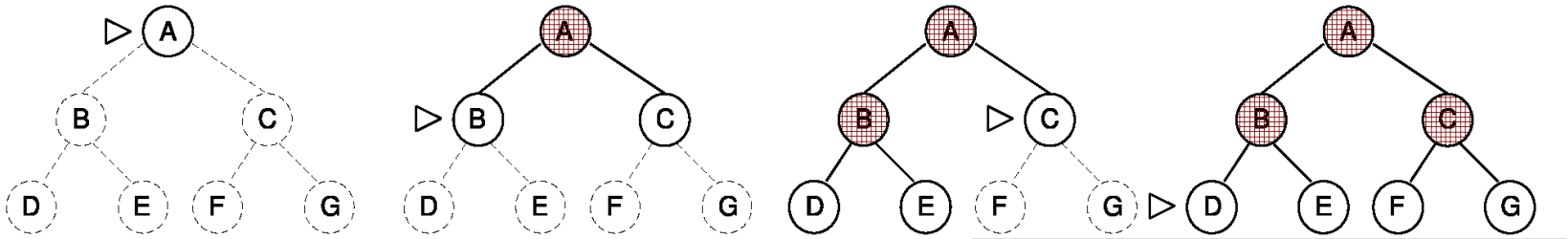
- **Estrategias**

Todas las estrategias se distinguen por el orden de expansión de los nodos.

Búsqueda primero en anchura

Se implementa llamando a la BÚSQUEDA-ÁRBOLES(problema,COLA-FIFO())

- Es completa. • El nodo objetivo más superficial no es necesariamente el óptimo
- Total de nodos generados $b + b^2 + b^3 + \ldots + b^d + (b^{d+1}-b) = O(b^{d+1})$.
- La complejidad en espacio es igual que la complejidad en tiempo.



Complejidad. Suponemos:

b=10

1 millón de nodos generados por segundo.

1000 bytes de almacenamiento para cada nodo.

nodo objetivo es el primero del nivel d.

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Estrategias sobre una repres. por espacio de estados

Búsqueda no informada o a ciegas

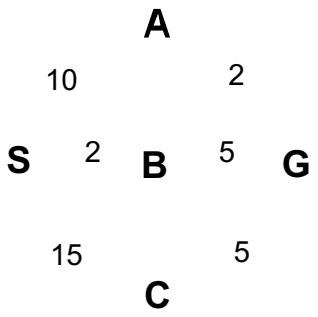
Búsqueda de costo uniforme

Se implementa BÚSQUEDA-ÁRBOLES(problema,frontera_O()) donde frontera_O está ordenada por g (costo del camino) de menor a mayor.

- Completa si el costo_individual $\geq \varepsilon$ (constante positiva pequeña).
- Esta condición es también suficiente para asegurar optimalidad.
- Complejidad en tiempo y espacio: $O(b^{[C^*/\varepsilon]})$, donde C* costo solución optima.

Árbol de búsqueda es infinito (aparecen ciclos). El método de búsqueda en árboles es poco eficiente en estos casos. En Costo Uniforme no hay problema.

Ir de S a G



Frontera
S(0)
B(2), A(10), C(15)
S(4), G(7), A(10), C(15)
B(6), G(7), A(10), A(14), C(15), C(19)
G(7), S(8), A(10), G(11), A(14), C(15), C(19)
S(8), A(10), G(11), A(14), C(15), C(19)

Solución = [S, B, G]

Coste del camino: g =7

Solución óptima costo mínimo

Estrategias sobre una repres. por espacio de estados

Búsqueda no informada o a ciegas

Búsqueda primero en profundidad

Esta estrategia puede implementarse por la BÚSQUEDA-ÁRBOLES con una cola último en entrar primero en salir (LIFO), también conocida como una pila (BÚSQUEDA-ÁRBOLES(problema,COLA-LIFO())).

- No es completa.
- No óptima.
- Total de nodos generados (complejidad en tiempo) $O(b^m)$.
- La complejidad en espacio O(bm) o O(m).

Búsqueda de profundidad limitada

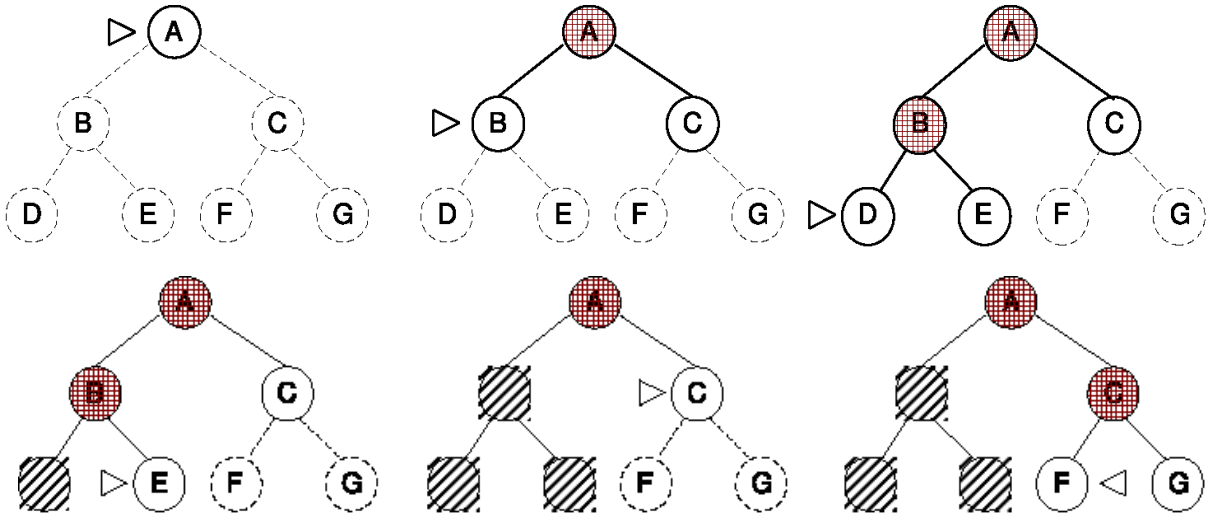
Búsqueda primero en profundidad con un límite de profundidad l predeterminado (introduce una fuente adicional de incompletitud). Se puede implementar con una simple modificación (para introducir l) del método de búsqueda en árboles.

- Si l<d no es completa ni óptima; Si l>d es completa pero no óptima.
- La complejidad en tiempo $O(b^l)$.
- La complejidad en espacio O(bl).

Estrategias sobre una repres. por espacio de estados

Búsqueda no informada o a ciegas

Profundidad



Estrategias sobre una repres. por espacio de estados

Búsqueda no informada o a ciegas

Búsqueda primero en profundidad con profundidad iterativa

La búsqueda con profundidad iterativa es una búsqueda en profundidad limitada que aumenta su límite hasta encontrar el objetivo (cuando alcanza d).

función BÚSQUEDA-PROFUNDIDAD-ITERATIVA(problema) **devuelve** una solución, o fallo

entradas: problema, un problema
para profundidad ← 0 a ∞ **hacer**
resultado ← BÚSQUEDA-PROFUNDIDAD-LIMITADA(problema,profundidad)
si resultado ≠ corte **entonces devolver** resultado

Combina las ventajas de la búsqueda en anchura (completitud, optimalidad) y búsqueda en profundidad (bajo uso de memoria).

Si *b* (factor de ramificación) es constante a todos los niveles, generar nodos repetidamente no es tan costoso.

Estrategias sobre una repres. por espacio de estados

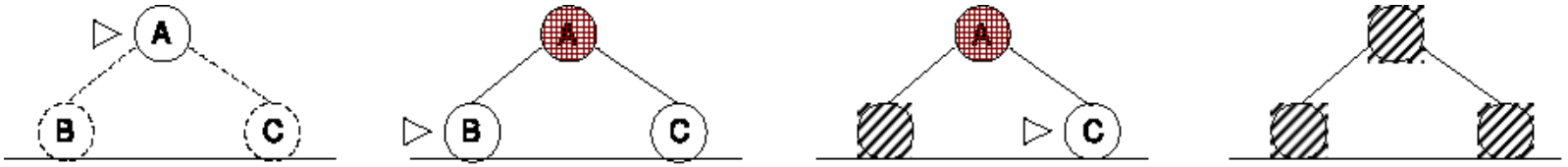
Búsqueda no informada o a ciegas

Profundidad iterativa

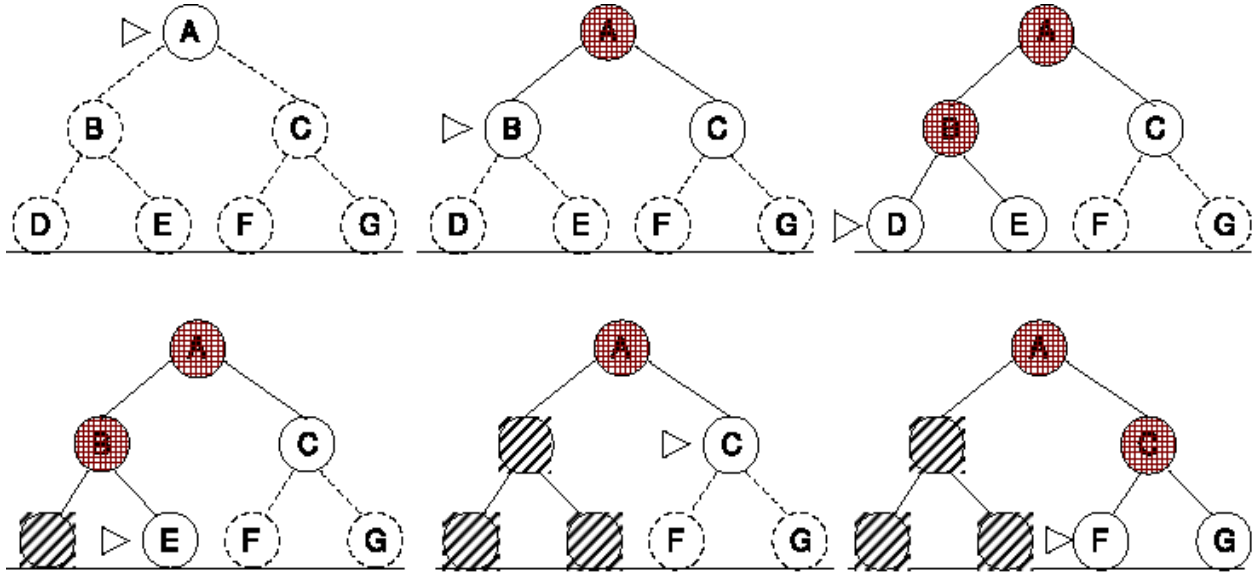
Límite=0



Límite=1



Límite=2



Estrategias sobre una repres. por espacio de estados

Búsqueda no informada o a ciegas

Comparando las estrategias de búsqueda no informada

Criterio	Primero en anchura	Costo uniforme	Primero en profundidad	Profundidad limitada	Profundidad iteratitva
¿completa?	Sí [1]	Sí [1,2]	No	No	Sí [1]
Tiempo	$O(b^{d+1})$	$O(b^{[C^*/\varepsilon]})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Espacio	$O(b^{d+1})$	$O(b^{[C^*/\varepsilon]})$	$O(bm)$	$O(bl)$	$O(bd)$
¿óptima?	Sí [3]	Sí	No	No	Sí [3]

donde:

b es el factor de ramificación;
d es la profundidad del nodo solución (objetivo) más superficial;
m es la longitud máxima de cualquier camino en el espacio de búsqueda;
l es el límite de profundidad.

[1] completa si b es finita;
[2] completa si los costos son $\geq \varepsilon$, para ε positivo;
[3] óptima si los costos son iguales.

Estrategias sobre una repres. por espacio de estados

Búsqueda no informada o a ciegas

- **Evitar estados repetidos – Búsqueda sobre grafos**

Para algunos problemas, árboles de búsqueda infinitos → si no se detectan los estados repetidos se puede provocar que un problema resoluble llegue a ser irresoluble. Un método que recuerde cada estado visitado → exploración directamente del grafo de espacio de estados.

- Crear una nueva estructura de datos (lista cerrada) que almacene los nodos expandidos (a veces a la frontera se le llama lista abierta).
- En problemas con muchos estados repetidos, la búsqueda en grafos es más eficiente que la búsqueda en árboles.

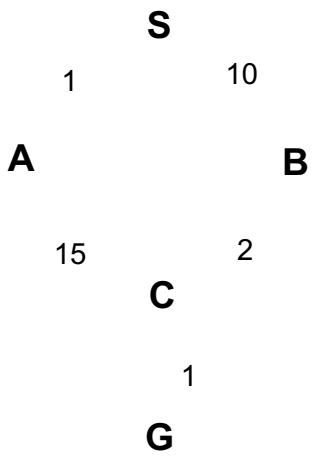
función BÚSQUEDA-GRAFOS(problema,frontera) **devuelve** una solución, o fallo

```
cerrado ← conjunto vacio
frontera ← INSERTAR(HACER-NODO(ESTADO-INICIAL[problema]),frontera)
bucle hacer
    si VACIA(frontera) entonces devolver fallo
    nodo ← BORRAR-PRIMERO(frontera)
    si TEST-OBJETIVO[problema](ESTADO[nodo]) entonces devolver SOLUCIÓN(nodo)
    si ESTADO[nodo] no esta en cerrado entonces
        añadir ESTADO[nodo] a cerrado
        frontera ← INSERTAR-TODO(EXPANDIR(nodo,problema),frontera)
```

Estrategias sobre una repres. por espacio de estados

Búsqueda no informada o a ciegas

Ejemplo: Ir de S a G (búsqueda en grafos, para costo uniforme).



Frontera	Cerrados
S(0)	
A(1), B(10)	S(0)
S(2), B(10), C(16)	S(0), A(1)
C(12), C(16), S(20)	S(0), A(1), B(10)
G(13), B(14), C(16), S(20), A(27)	S(0), A(1), B(10), C(12)
B(14), C(16), S(20), A(27)	S(0), A(1), B(10), C(12),G(13)

Solución=[S,B,C,G]. Costo del camino: g=13 (sol. óptima)

Complejidad: tiempo=6 nodos, memoria=9 nodos.

Estrategias sobre una repres. por espacio de estados

BÚSQUEDA HEURÍSTICA

- **Introducción**

Las estrategias de búsqueda no informadas encuentran soluciones generando sistemáticamente nuevos estados y probándolos con el test objetivo



Tremendamente ineficientes en problemas difíciles



Alternativa: disponer de algún mecanismo (heurística) que permita dirigir la búsqueda hacia las zonas más prometedoras, y encontrar soluciones de una manera más eficiente

Las heurísticas (como ya hemos comentado), en un sentido general, son criterios, reglas, puntuaciones o métodos que ayudan a decidir cuál es la mejor alternativa o más prometedora, entre varias posibles, para alcanzar un determinado objetivo.

Este mecanismo requiere disponer de conocimiento sobre el problema.

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

En el contexto de los sistemas de búsqueda, las heurísticas se suelen utilizar para decidir:

- qué nodo expandir a continuación (más prometedor entre candidatos).
- qué nodos sucesores son generados (algunos podrían estar repetidos).
- qué nodos nunca deberían tenerse en cuenta (diferentes tipos de poda).

Las estrategias de **búsqueda informada** o **búsqueda heurística** son:

- Estrategias de búsqueda que utilizan algún mecanismo heurístico para dirigir la búsqueda hacia las zonas más prometedoras y encontrar soluciones de una manera más eficiente.
- Existen diversas estrategias, que se basan en los métodos básicos y subyacentes para la búsqueda no informada.
- La búsqueda informada puede encontrar soluciones de una manera más eficiente que una búsqueda no informada.

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

- **Estrategias**

La búsqueda primero el mejor es la estrategia general y más básica de búsqueda informada.

Es un caso particular del método general de BÚSQUEDA-ÁRBOLES o BÚSQUEDA-GRAFOS.

Se utiliza una **función de evaluación** f(n) que, para cada nodo n, devuelve un valor numérico indicando lo prometedor que es el nodo para ser expandido.

- Este valor se usa para ordenar la lista frontera, de forma que los nodos más prometedores estén al principio.
- Tradicionalmente, el nodo más prometedor es el nodo con evaluación más baja → se interpreta como distancia al objetivo.

Puede implementarse dentro de nuestro marco general de búsqueda con una cola con prioridad (orden ascendente de f-valores).

Hay una familia de métodos de Búsqueda primero el mejor que usan funciones de evaluación diferentes.

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Una componente clave es el cálculo de una **función heurística**, h:

- h(n) es el coste estimado del camino más barato desde el nodo n a un nodo objetivo, y
- si n es un nodo objetivo, entonces h(n)=0.

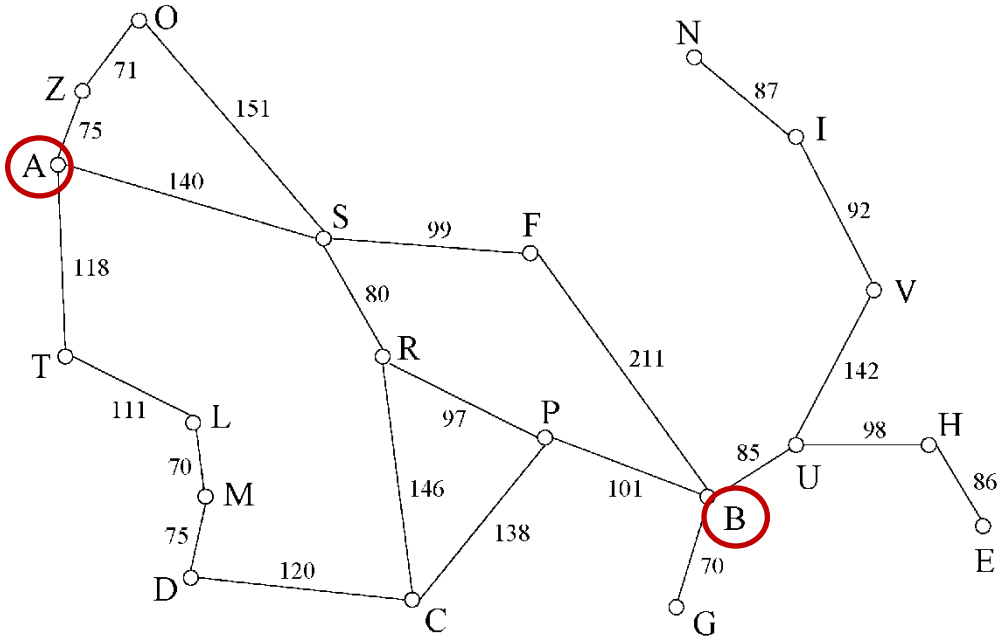
Las funciones heurísticas son la forma más común de transmitir el conocimiento adicional del problema al método de búsqueda.

Ejemplo: Mapa de carreteras

$h(n)=h_{DLR}(n)$

A 366	B 0	C 160	D 242
E 161	F 176	G 70	H 151
I 226	L 244	M 241	N 234
O 380	P 100	R 193	S 253
T 329	U 80	V 199	Z 374

Estimar el coste del camino más barato desde cualquier nodo hasta B con la distancia en línea recta.



Estrategias sobre una repres. por espacio de estados

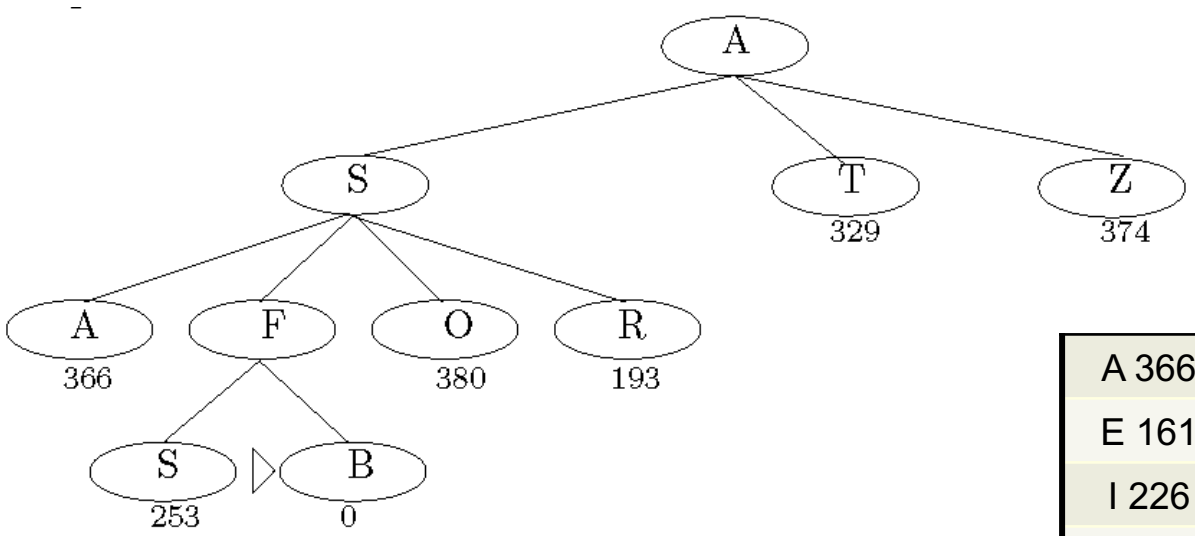
Búsqueda heurística

Búsqueda primero el mejor avara o voraz (greedy)

La búsqueda primero el mejor avara expande el nodo que estima más cercano al objetivo. Evalúa los nodos mediante f(n) = h(n).

Complejidad tiempo y espacio → $O(b^m)$.

Con una buena función h, se reduce la complejidad considerablemente. La cantidad de la reducción depende del problema y de la heurística.



Camino desde A a B usando $h(n)=h_{DLR}(n)$

A 366	B 0	C 160	D 242
E 161	F 176	G 70	H 151
I 226	L 244	M 241	N 234
O 380	P 100	R 193	S 253
T 329	U 80	V 199	Z 374

Solución encontrada: A-S-F-B, costo 450 (no óptima)
Complejidad: tiempo=4 nodos, memoria= 10 nodos

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Búsqueda A: minimizando el costo estimado total de la solución*

Evalúa los nodos mediante $f(n)=g(n)+h(n)$

g(n): coste del camino desde inicio a n;

h(n): coste estimado del camino más barato desde n al objetivo.

f(n):coste estimado del camino más barato a la solución a través de n

Si la función heurística h satisface ciertas condiciones, la búsqueda A* es:

completa

óptima

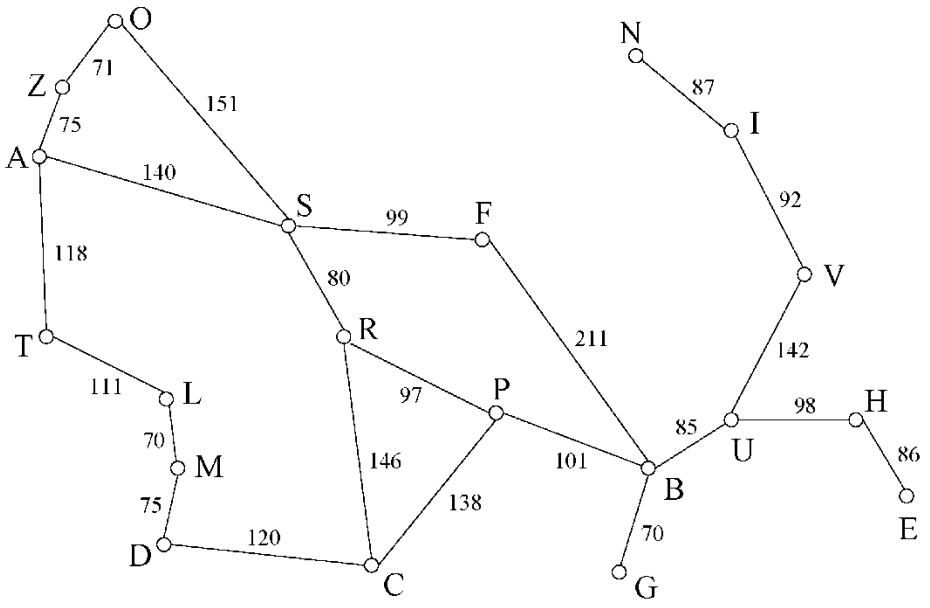
Estrategias sobre una repres. por espacio de estados

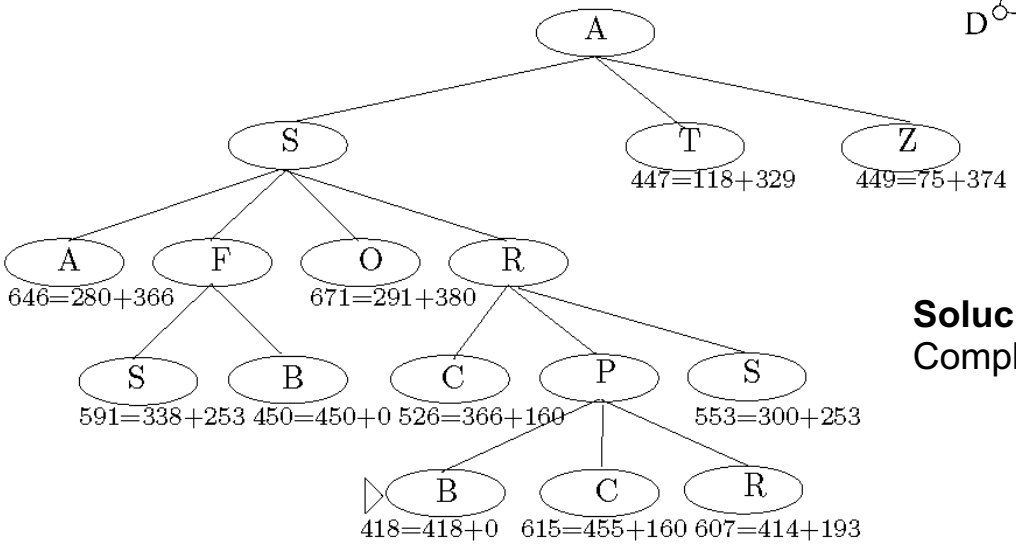
Búsqueda heurística

Ejemplo: $f(n) = g(n) + h_{DLR}(n)$

h_{DLR} función heurística

A 366	B 0	C 160	D 242
E 161	F 176	G 70	H 151
I 226	L 244	M 241	N 234
O 380	P 100	R 193	S 253
T 329	U 80	V 199	Z 374





Solución encontrada: A-S-R-P-B, costo 418
Complejidad: tiempo=6 nodos, memoria= 15 nodos

¿Óptima?

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Una heurística h() es **admisible** si nunca sobrestima el menor coste posible de alcanzar el objetivo, es decir,

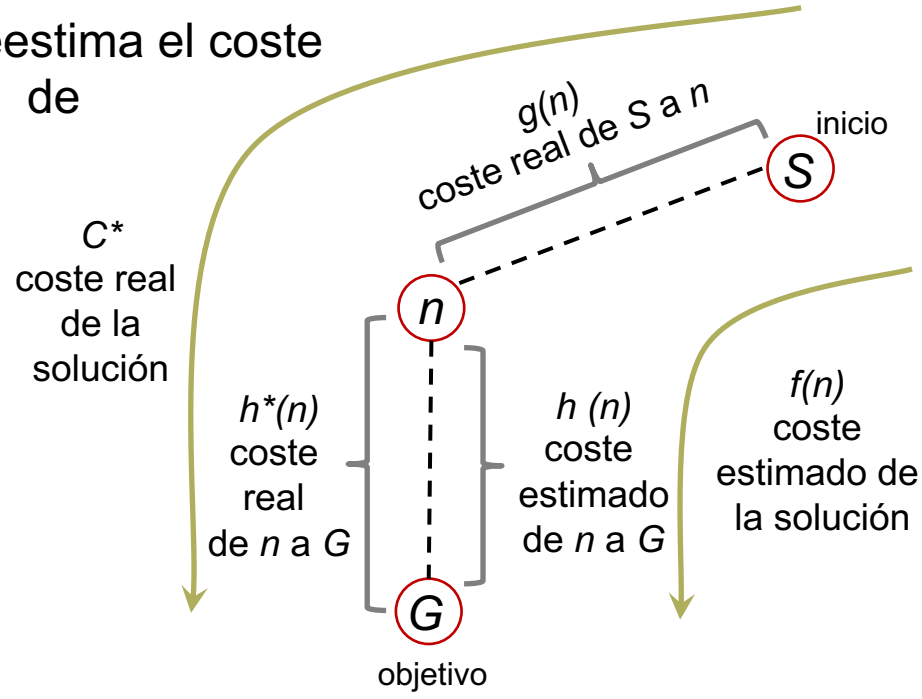
$$h(n) \leq h^*(n), \quad \forall\ n$$

donde h*(n) es el menor coste posible de alcanzar el objetivo desde el nodo n.

Consecuencia:

- Como f(n)=g(n)+h(n), y g(n) es el coste real de alcanzar n a través del camino actual, f(n) nunca sobreestima el coste verdadero de la solución a lo largo de un camino que pasa por n.

$$f(n) \leq C^*$$



Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Las heurísticas no admisibles son pesimistas:

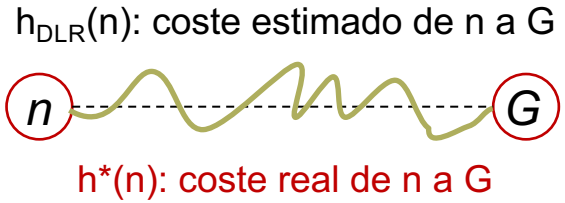
- Pueden descartar un sucesor porque "piensan" que es peor de lo que realmente es.
- Esto impide la optimalidad: pueden perder soluciones óptimas.

Las heurísticas admisibles son optimistas:

- Suponen que el coste de resolver un problema es menor del que realmente es.
- Esto permite la optimalidad: un nodo en el camino óptimo no puede parecer tan malo como para descartarlo.

La heurística de la distancia en línea recta, h_{DLR}, es una heurística admisible:

- La distancia en línea recta es la distancia más corta posible entre dos puntos: siempre será menor que la distancia real por carretera.



Optimalidad de A* para BÚSQUEDA-ÁRBOLES

Teorema: si la función h es una heurística admisible entonces la búsqueda A* guiada por h es óptima.

Si h es admisible entonces A* es óptima

(como g(n) es el coste exacto para alcanzar n y h(n) no sobreestima h*(n), f(n) nunca sobrestima el coste verdadero de una solución a través de n)

Demostración:

Sea C* el coste de la solución óptima. Supongamos que tenemos en la frontera un nodo objetivo sub-óptimo G_2 .

- Como G_2 es sub-óptimo y $h(G_2)=0$ (por ser nodo objetivo):

$$f(G_2) = g(G_2)+h(G_2) = g(G_2) > C^*$$

- Consideremos un nodo n en la frontera sobre un camino solución óptimo. Por ser h admisible:

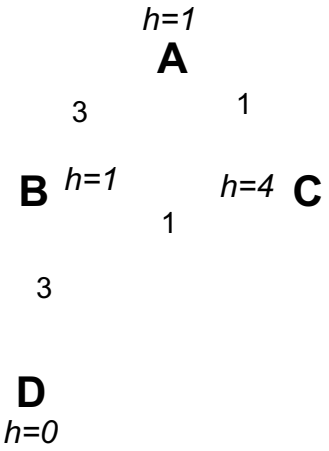
$$f(n) = g(n)+h(n) \leq g(n)+h^*(n) = C^*$$

- Por tanto, $f(n) \leq C^* < f(G_2)$. En consecuencia, escogerá n en lugar de G_2. A* devolverá solución óptima.

Optimalidad de A* para BÚSQUEDA-GRAFOS

- La admisibilidad no es suficiente para garantizar optimalidad.
  - La demostración falla y el método puede devolver soluciones sub-óptimas.
  - Motivo: BÚSQUEDA-GRAFOS puede ignorar el camino óptimo si aparece un estado repetido y se descarta por estar ya en cerrados.

Demostración por contraejemplo:



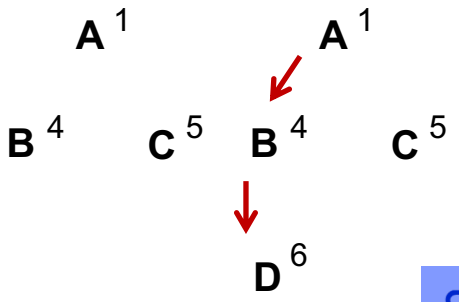
Óptimo A-C-B-D (5)

h admisible

h(n)≤h*(n), ∀n

n	h(n)	h*(n)
A	1	5
B	1	3
C	4	4
D	0	0

Frontera	Cerrado
A(1)	
$B_A(4),C_A(5)$	A(1)
$C_A(5),D_B(6),A_B(7),C_B(8)$	$A(1),B_A(4)$
~~$A_C(3),B_C(3)$~~,$D_B(6),A_B(7),C_B(8)$	$A(1),B_A(4),C_A(5)$
$A_B(7),C_B(8)$	$A(1),B_A(4),C_A(5),D_B(6)$



Solución: A-B-D (6) no óptima

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Hemos probado que la BÚSQUEDA-GRAFOS falla, descartando nodos repetidos mejores y, por tanto, posiblemente soluciones óptimas.

Ante esto, tenemos dos posibilidades:

1.- Extender la BÚSQUEDA-GRAFOS de modo que deseche el camino más caro de cualquiera dos caminos encontrado al mismo nodo. Cálculo complementario complicado (menos eficiente) pero garantiza la optimalidad.

2.- Exigir alguna propiedad, más estricta que la admisibilidad, que asegure que el camino óptimo a cualquier estado repetido es siempre el primero que seguimos -- como en el caso de la búsqueda de costo uniforme.

La posibilidad 2 (asegurar que el camino óptimo a cualquier estado repetido es siempre el primero que seguimos) se mantiene si imponemos una nueva condición a h(n): **Condición de consistencia.**

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Condición de consistencia: una función heurística h es consistente sii

$$h(n) \leq K(n,a_s,n') + h(n'), \quad \forall\ n, n'$$

donde $K(n,a_s,n')$ costo camino de n y n' a través de secuencia de acciones a_s.

Es una condición ligeramente más estricta que la admisibilidad ($h(n) \leq h^*(n), \forall n$).

Condición de monotonía: Una función heurística h es monótona sii

$$h(n) \leq c(n,a,n') + h(n'), \forall\ n, n', \text{ con } n' \text{ sucesor inmediato de } n$$

donde c(n,a,n') costo del arco que une n y n'.

Se puede demostrar que:

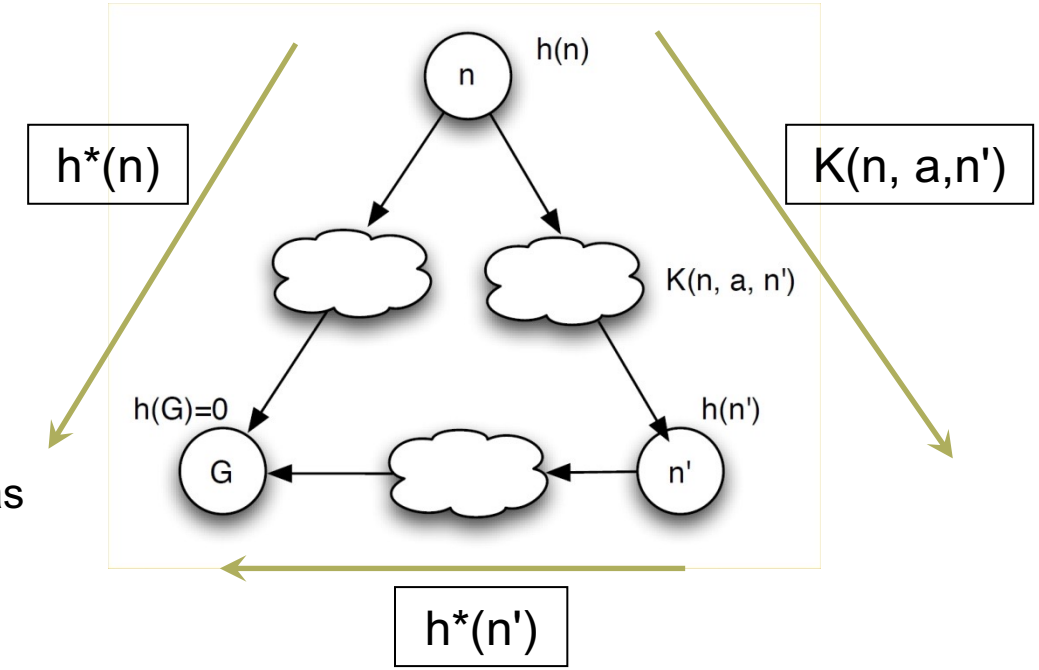
h es **consistente** $\Leftrightarrow$ h es **monótona**

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Lo costes reales, h*(n), considerados como distancias, deben cumplir la propiedad de la desigualdad triangular.

Ningún lado de un triángulo tiene una longitud mayor que la suma de las longitudes de los otros dos lados



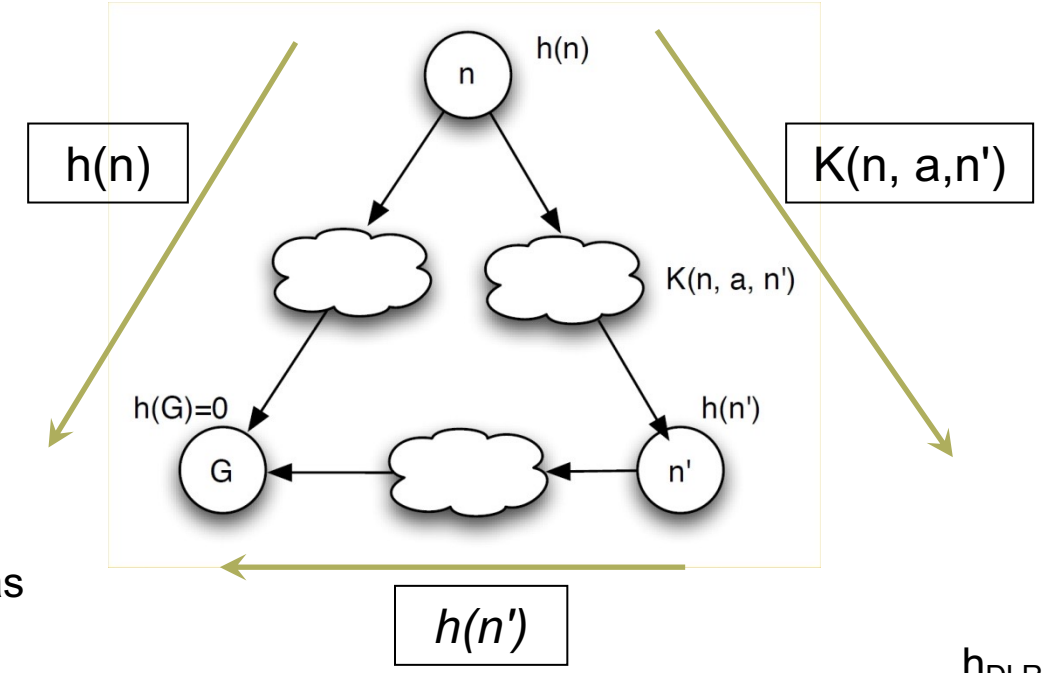
G: nodo objetivo más cercano a n

$$h^*(n) \leq K(n,a_s,n') + h^*(n'), \quad \forall\ n, n'$$

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Lo que la **condición de consistencia establece es que se cumpla la propiedad** de la desigualdad triangular también para los costes estimados, h.



G: nodo objetivo más cercano a n

$$h(n) \leq K(n,a_s,n') + h(n'), \quad \forall\ n, n'$$

h_{DLR} es una heurística consistente por ser una distancia métrica.

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Teorema: Si h es una heurística consistente entonces h es admisible.

h consistente $\Rightarrow$ h admisible

Demostración:

Sea un nodo n cualquiera, y G el objetivo más cercano.

Por ser h consistente, tenemos que $h(n) \leq K(n,a_s,G) + h(G)$.

Costo del camino que une n a G

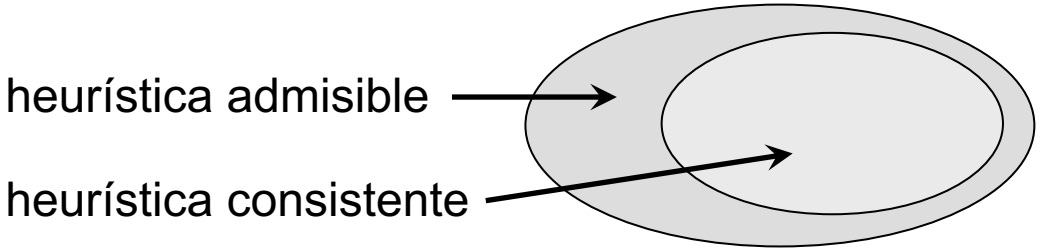
Y como $h(G) = 0$ y $K(n,a_s,G) = h^*(n)$, concluimos que:

Costo real del camino

$$h(n) \leq h^*(n), \forall n$$

Luego h es admisible.

Lo contrario no es cierto. Una heurística admisible no siempre es consistente.



Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Las conclusiones más importantes de la condición de consistencia son:

a) Si h es consistente entonces los valores de f, a lo largo de cualquier camino, no disminuyen.

Sea n' un nodo en el camino desde n entonces

$$g(n')=g(n)+K(n,a_s,n') \text{ para una secuencia de acciones } a_s$$

por tanto, $f(n') = g(n')+h(n') = g(n)+\underbrace{K(n,a_s,n')+h(n')}_{\geq \text{ por consistencia}} \geq g(n)+h(n) = f(n).$

Con lo cual, la secuencia de nodos expandidos por A* utilizando la BÚSQUEDA-GRAFOS están en orden no decreciente de f(n).

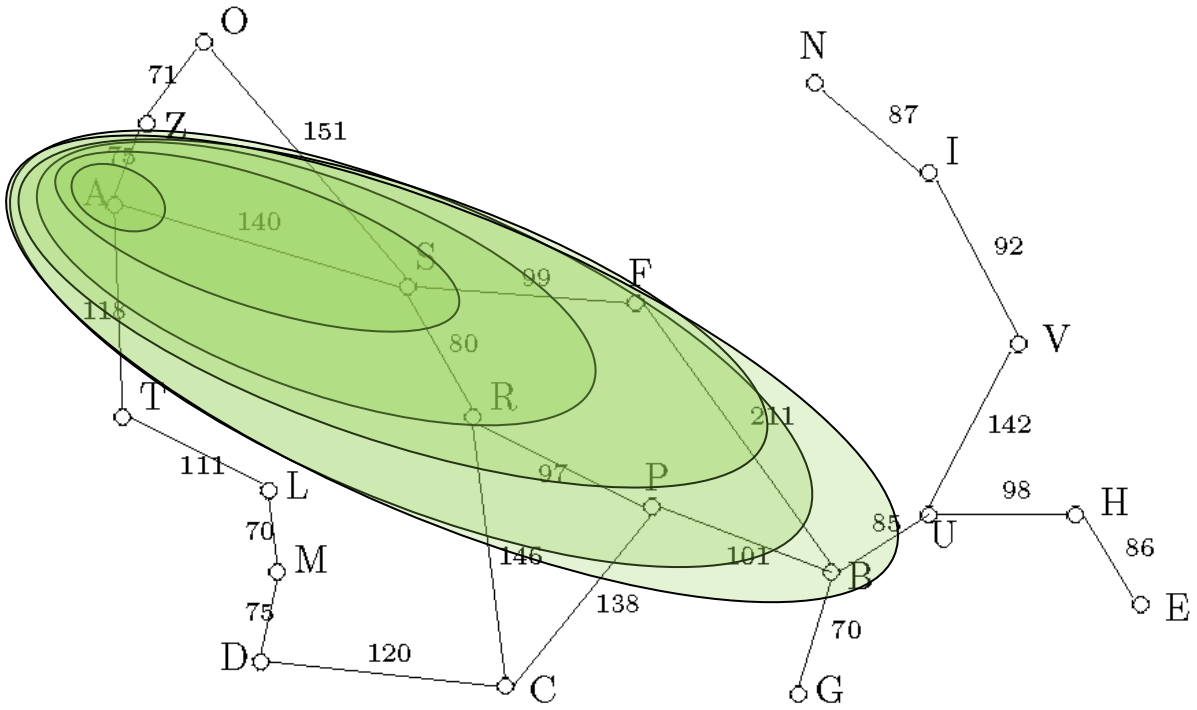
El hecho que los f-costos no disminuyan a lo largo de cualquier camino significa que podemos dibujar curvas de nivel en el espacio de estados, como las curvas de nivel en un mapa topográfico.

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Debido a que A* expande el nodo de frontera de f-coste más bajo (con f siempre creciente) podemos ver que A* busca hacia fuera desde el nodo inicial, añadiendo nodos en bandas concéntricas de f-coste.

Dentro de la curva de nivel etiquetada con 417, todos los nodos tienen la f(n) menor o igual a 417, etc.



f(S)=393, f(R)=413, f(F)=415, f(P)=417

b) Si h es consistente entonces A* utilizando la BÚSQUEDA-GRAFOS es óptimo.

Si h es consistente entonces, cada vez que A* selecciona un nodo para expandir, el camino óptimo a ese nodo ha sido encontrado. Como consecuencia, el primer nodo objetivo seleccionado para la expansión debe ser una solución óptima.

Teorema: si h es una heurística consistente entonces la búsqueda A* guiada por h es óptima.

Demostración:

Sea C* el coste de la solución óptima G. Supongamos que tenemos en la frontera un nodo objetivo sub-óptimo G_2.

- G_2 es sub-óptimo y $h(G_2)=0$ (nodo objetivo): $f(G_2) = g(G_2)+h(G_2) = g(G_2) > C^*$
- Sea un nodo n en la frontera sobre el camino solución óptimo. Por ser h(n) consistente $f(n) = g(n)+h(n) \leq g(n)+k(n,G)+h(G) = g(G)+h(G) = g(G) = C^*$

Por tanto, $f(n) \leq C^* < f(G_2)$. En consecuencia, escogerá n en lugar de G_2. A* devolverá solución óptima.

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Recapitulando sobre la optimalidad de A*:

- Si el espacio de estados es un árbol y podemos usar BÚSQUEDA-ÁRBOLES, entonces A* con una heurística **admisible** garantiza una solución óptima.
- Si el espacio de estados es un grafo y debemos usar BÚSQUEDA-GRAFOS, entonces A* con una heurística **consistente** garantiza una solución óptima.
- Si una heurística es **consistente**, entonces es admisible. Lo contrario no es cierto.

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

La búsqueda A* es:

completa, óptima, y óptimamente eficiente entre todos los métodos.

Lamentablemente, no significa que A* sea la respuesta a todas nuestras necesidades de búsqueda.

Dificultad → para la mayoría de los problemas, el número de nodos dentro de la curva de nivel del objetivo en el espacio de búsqueda es todavía exponencial en la longitud de la solución.

El tiempo computacional no es, sin embargo, la desventaja principal de A*.

Como mantiene todos los nodos generados en memoria, A*, por lo general, se queda sin mucho espacio antes de que se quede sin tiempo.

Hay métodos posteriores que han vencido el problema de espacio sin sacrificar la optimalidad o la completitud, con un pequeño coste en el tiempo de ejecución.

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Búsqueda primero el mejor recursiva (BPMR) o con memoria acotada

Limitar el consumo de memoria en la búsqueda heurística.

BPMR imita a Primero el Mejor pero genera menos nodos y el consumo de memoria es lineal. Evita profundizar sin límite por un camino que podría empezar a empeorar.

- Usa una variable $f_{límite}$ *para recordar el* f-valor del mejor camino alternativo desde cualquier ancestro del nodo actual.
- Si el nodo actual excede este límite, la recursión retrocede al camino alternativo, descargando el subárbol actual.
- Al retroceder, el método reemplaza el *f-valor* de cada nodo a lo largo del camino con un valor de respaldo: el mejor *f-valor* de sus hijos.

La idea general es recordar el mejor *f-valor* del subárbol olvidado, para decidir si merece la pena re-expandir el subárbol en algún momento posterior.

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

función BPMR(problema) **devuelve** una solución, o fallo

BPMR-f(problema,HACER-NODO(ESTADO-INICIAL[problema]),∞)

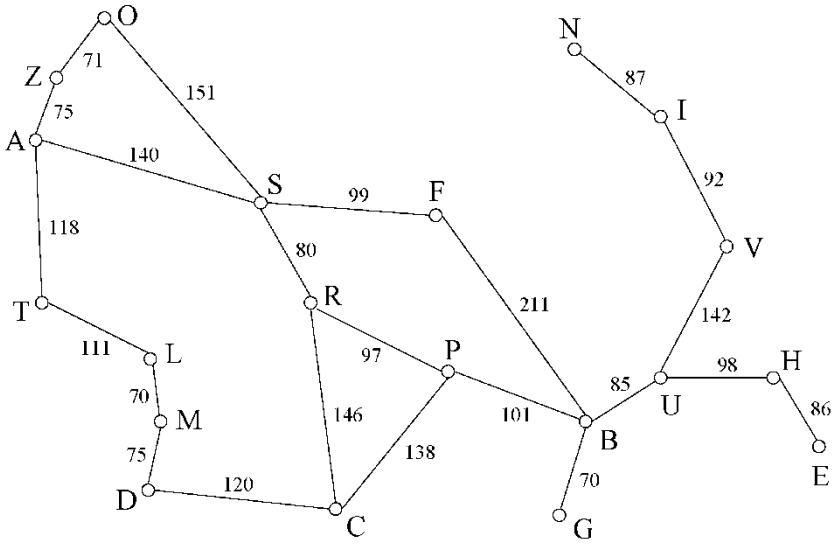
función BPMR-f(problema,nodo,$f_{límite}$) **devuelve** una solución o fallo y un nuevo $f_{límite}$

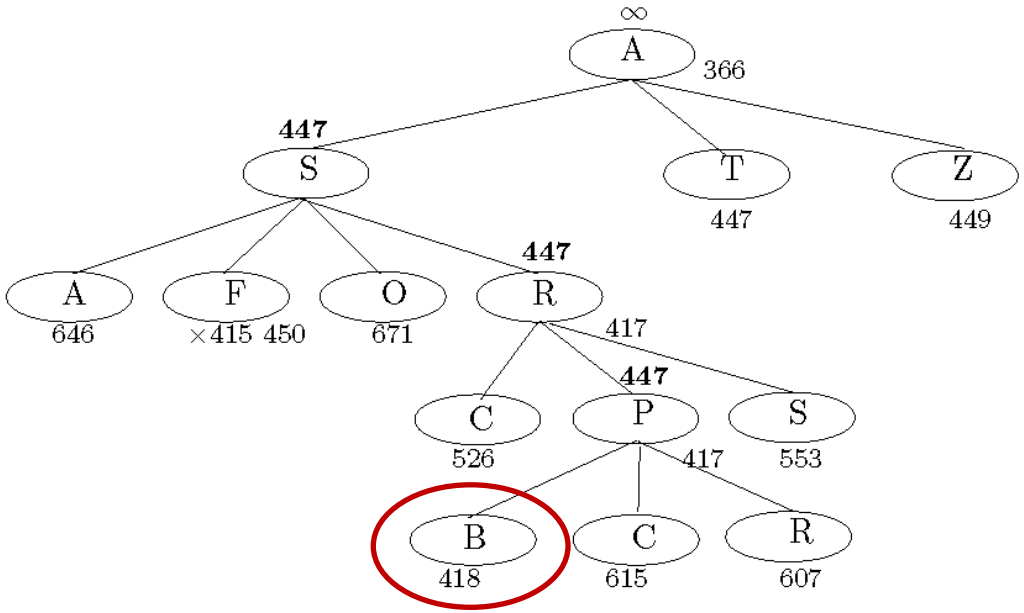
si TEST-OBJETIVO[problema](estado) **entonces devolver** nodo
sucesores ← EXPANDIR(nodo,problema)
si sucesores está vacío **entonces devolver** fallo, ∞
para cada s en sucesores **hacer** f[s]← max(g(s)+h(s) , f[nodo])
repetir
mejor ← el nodo con el f-valor más pequeño de sucesores
si f[mejor] > f_{limite} **entonces devuelve** fallo, f[mejor]
alternativa ← el nodo con el segundo f-valor más pequeño entre los sucesores
resultado,f[mejor] ← BPMR-f(problema,mejor,min($f_{límite}$,alternativa))
si resultado ≠ fallo **entonces devolver** resultado

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Estado inicial: A; Estado Final: B





A 366	B 0	C 160	D 242
E 161	F 176	G 70	H 151
I 226	L 244	M 241	N 234
O 380	P 100	R 193	S 253
T 329	U 80	V 199	Z 374

Solución encontrada: A-S-R-P-B

costo 418 -- óptima

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

BPMR es una método óptimo si la función h verifica las mismas condiciones de optimalidad que A* (admisible en BÚSQUEDA-ÁRBOLES y monótona en BÚSQUEDA-GRAFOS).

Su complejidad en espacio es O(bd), pero su complejidad en tiempo es bastante difícil de caracterizar: depende de la exactitud de la función heurística y de cómo cambia a menudo el mejor camino mientras se expanden los nodos.

Por tanto, BPMR:

- utiliza un espacio lineal, debido a que sólo mantiene un árbol en memoria.
- consume mucho tiempo porque debe reexpandir los mismos nodos varias veces.

Si hubiese más memoria disponible, sería más sensato aplicar un método que use toda la memoria disponible a cambio de disminuir la complejidad en tiempo.

Varios métodos: uno de ellos A*MS (A* con memoria acotada simplificada).

A*MS avanza como A*, expandiendo la mejor hoja hasta que la memoria este llena. En este punto, no se puede añadir un nuevo nodo al árbol de búsqueda sin retirar uno viejo. A*MS siempre retira el peor nodo hoja (el de f-valor más alto).

Estrategias sobre una repres. por espacio de estados

Búsqueda heurística

Como en BPMR, A*MS devuelve hacia atrás, a su padre, el valor del nodo olvidado. De este modo, el antepasado de un subárbol olvidado sabe la calidad del mejor camino en el subárbol. Con esta información, A*MS vuelve a generar el sub-árbol sólo cuando todos los caminos parecen peores que el camino olvidado.

A*MS es completo si hay alguna solución alcanzable - es decir, si d es menor que el tamaño de memoria (expresada en nodos).

Es óptimo si cualquier solución óptima es alcanzable; de otra manera devuelve la mejor solución alcanzable. En términos prácticos, A*MS podría ser el mejor método de uso general para encontrar soluciones óptimas.

Una forma de perder la optimalidad pero de una manera controlada es la siguiente:

Podemos incrementar en una cantidad "e" el salto de la frontera de optimalidad de manera que si $h^*(n) < h(n)$, obtengamos $h(n) \leq h^*(n)+e$.

(posiblemente, para determinados problemas, solamente podremos diseñar, de una manera efectiva, heurísticas que no son admisibles)

Se denominan **heurísticas e-admisibles.** Son importantes porque obtienen **soluciones e-optimas**, es decir, que no es óptima por tan solo una cantidad "e".

ESTRATEGIAS SOBRE UNA REPRESENTACIÓN POR REDUCCIÓN

Al igual que en la sección anterior, realizamos el estudio de esta nueva sección siguiendo el siguiente esquema:

- Métodos de búsqueda
  - sin información o a ciegas
    - Anchura
    - Profundidad
  - con información o heuristicos
    - YO*

Estrategias sobre una representación por reducción

BÚSQUEDA NO INFORMADA O A CIEGAS

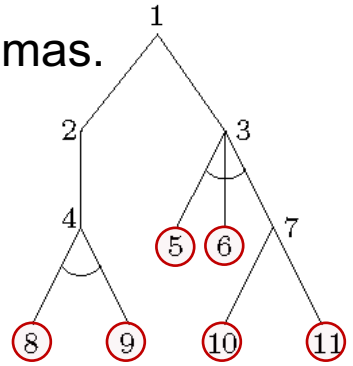
- **Introducción**

Inicialmente utilizaremos un árbol de búsqueda YO (más adelante lo generalizaremos) para resolver un problema. Para ello, especificamos:

- un nodo comienzo,
- conjunto de nodos terminales (primitivos) y
- conjunto de operadores para reducir el problema en sub-problemas.

Nodo O: La aplicación de un operador a un nodo n se representa por un arco dirigido desde el nodo n al nodo sucesor.

- Reduce el problema a varios sub-problemas alternativos mas simples.
- Para resolver el problema, basta resolver unos de ellos.



5,6,8,9,10,11 nodos terminales

Nodo Y: La aplicación de un operador a un nodo n se representa por un conjunto de arcos dirigidos desde el nodo n a un conjunto de nodos sucesores.

- Reduce el problema a un conjunto de subproblemas más simples.
- Para resolver el problema, hay que resolver todos ellos.

Estrategias sobre una representación por reducción

Búsqueda no informada o a ciegas

Una solución al problema original esta dado por un subárbol del árbol YO suficiente para demostrar que el nodo comienzo es resoluble.

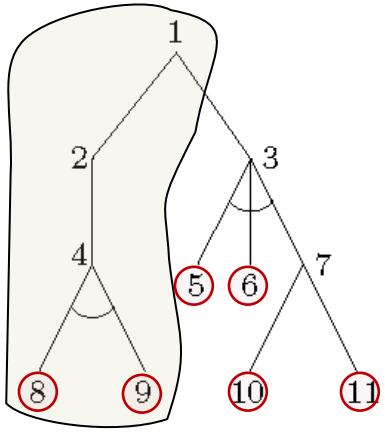
Características en común con los métodos de búsqueda en espacio de estados:

La operación de expandir está también presente.

Los métodos difieren principalmente en el orden de expansión.

Hacemos dos suposiciones simplificadoras:

- El espacio de búsqueda es un árbol YO y no un grafo YO general, y
- Cuando un problema se transforma en un conjunto de sub-problemas, estos se pueden resolver en cualquier orden.

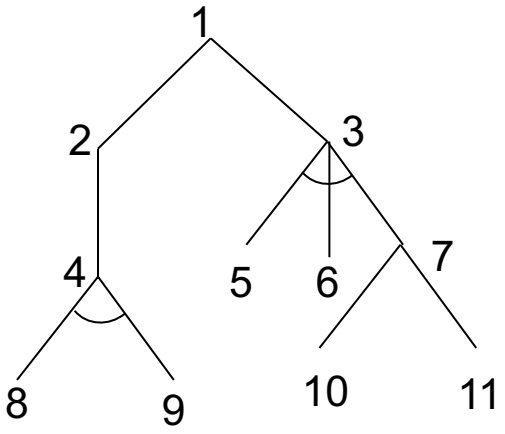


5, 6, 8, 9, 10 y 11 nodos terminales

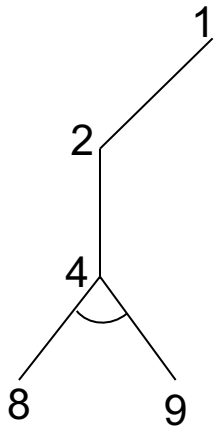
Estrategias sobre una representación por reducción

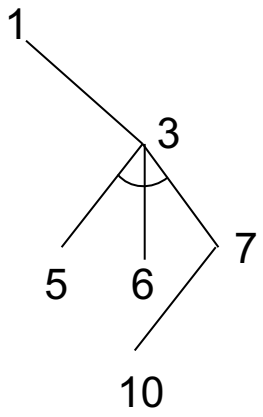
Búsqueda no informada o a ciegas

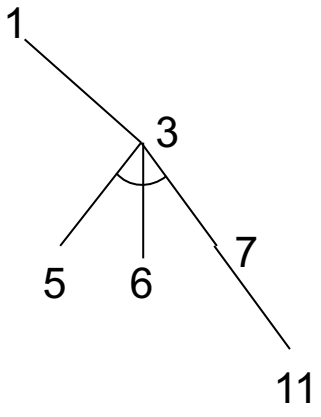
Ejemplo de soluciones de un árbol YO:



- Hay tres posibles soluciones (subárboles solución):







Estrategias sobre una representación por reducción

Búsqueda no informada o a ciegas

- **Búsqueda sobre árboles**

función BÚSQUEDA-ÁRBOLES-YO(problema,frontera) **devuelve** una solución, o fallo

```
frontera ← INSERTA(HACER-NODO(PROBLEMA-INICIAL[problema],frontera)
bucle hacer
    si VACIA(frontera) entonces devolver fallo
    nodo ← BORRAR-PRIMERO(frontera)
    frontera ← INSERTAR-TODO(EXPANDIR-YO(nodo,problema),frontera)
    si EXPANDIR-YO(nodo,problema)={ } entonces
        nodo ← irresoluble
        si nodo hace irresoluble alguno de sus ancestros entonces ancestros ← irresolubles
        si PROBLEMA-INICIAL=irresoluble entonces fallo. STOP
        BORRAR(frontera) nodos con ancestros irresolubles
    en otro caso, si se han generado nodos resolubles entonces
        etiquetar esos nodos como resolubles
        si esos nodos hacen resolubles a algunos de sus ancestros entonces ancestros ← resolubles
        si PROBLEMA-INICIAL=resoluble entonces devolver SOLUCIÓN
        BORRAR(frontera) cualquier nodo etiquetado como resoluble o que tenga ancestros resolubles
```

función EXPANDIR-YO(nodo,problema) **devuelve** un conjunto de nodos

```
SUC ← conjunto vacio
para cada sucesor, m, de nodo
    si m es un conjunto de subproblemas entonces
        SUC ← INSERTAR(todos los sucesores de m)
    en otro caso SUC ← INSERTAR(m)
devolver SUC
```

Estrategias sobre una representación por reducción

Búsqueda no informada o a ciegas

- **Estrategias**

Las estrategias se distinguen por el orden de expansión de los nodos.

Búsqueda Primero en Anchura en Árboles YO

Se implementa llamando a BÚSQUEDA-ÁRBOLES-YO(problema,COLA-FIFO())

Búsqueda Primero en Profundidad en Árboles YO

Se implementa llamando a BÚSQUEDA-ÁRBOLES-YO(problema,COLA-LIFO())

Búsqueda de Profundidad limitada en Árboles YO

Se puede aliviar el problema de los árboles ilimitados aplicando la búsqueda primero en profundidad con un límite de profundidad predeterminado.

El límite de profundidad también introduce una fuente adicional de incompletitud, puede que nos quedemos sin resolver el problema principal.

Estrategias sobre una representación por reducción

BÚSQUEDA HEURÍSTICA

- **Introducción**

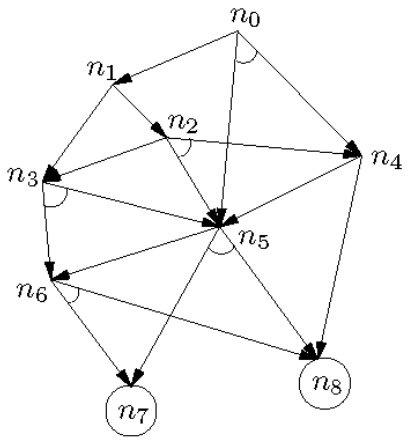
Ahora vamos a tratar el problema de la búsqueda heurística en un grafo YO.

La presencia de nodos Y añaden complicaciones conceptuales.

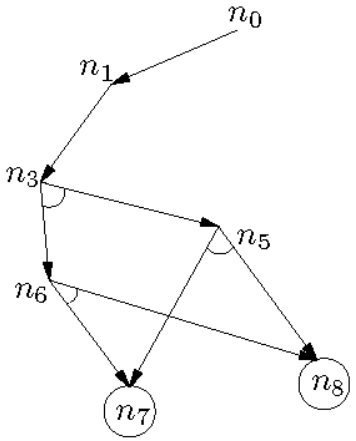
Grafos YO → Hipergrafos.

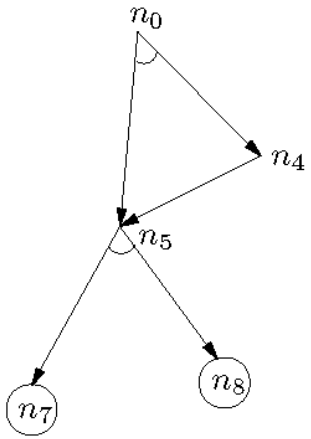
Arcos que conectan pares de nodos → hiperarcos que conectan un nodo antecesor con un conjunto de nodos sucesores.

Conectores → hiperarcos (k-conector dirigido desde un nodo a un conjunto de k nodos sucesores).



Soluciones de n_0 a $\{n_7, n_8\}$





Estrategias sobre una representación por reducción

Búsqueda heurística

Podemos dar una definición recursiva precisa de un grafo solución. La definición supone que nuestros grafos YO no poseen ciclos.

Designemos con G' un **grafo solución** desde el nodo n a un conjunto N de nodos en un grafo YO, G.

G' es un subgrafo de G.

- Si n es un elemento de N, G' consta tan solo del nodo n;
- en otro caso,
  - si n tiene un conector k, que parte de él, dirigido a los nodos $\{n_1,\ldots,n_k\}$, tales que hay un grafo solución hasta N desde cada uno de los n_i con i=1,…,k, se verifica que G' consta:
    - del nodo n,
    - del conector k,
    - de los nodos $\{n_1,\ldots,n_k\}$, y
    - de los grafos solución a N desde cada uno de los nodos $\{n_1,\ldots,n_k\}$;
  - en caso contrario, no hay grafo solución desde n a N.

Estrategias sobre una representación por reducción

Búsqueda heurística

De forma análoga a como se asignan costos a los arcos de un grafo ordinario, resulta conveniente asignar costos a los conectores de los grafos YO.

Los costos de los conectores pueden usarse entonces para calcular **el costo de un grafo solución**.

Designemos con K(n,N) el costo de un grafo solución desde el nodo n a N.

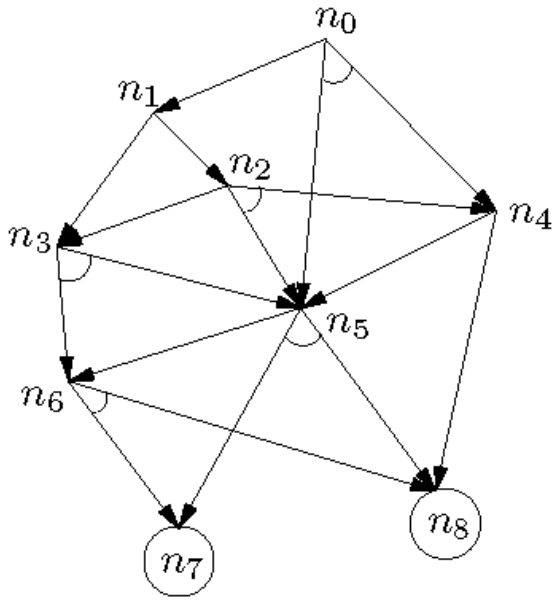
- Si n es un elemento de N, K(n,N)=0
- en otro caso,
  - si n tiene un conector que parte de él hacia un conjunto de nodos sucesores $\{n_1,\ldots,n_k\}$, en el grafo solución; sea c_n el costo de ese conector, entonces $K(n,N)=c_n+K(n_1,N)+\ldots+K(n_k,N)$

NOTA: esta definición de costo de un grafo solución puede contar más de una vez los costos de algunos de sus conectores. Por ejemplo, suponiendo que el costo de cada k-conector es k, los costos de los grafos solución de las figuras son, respectivamente, 8 y 7.

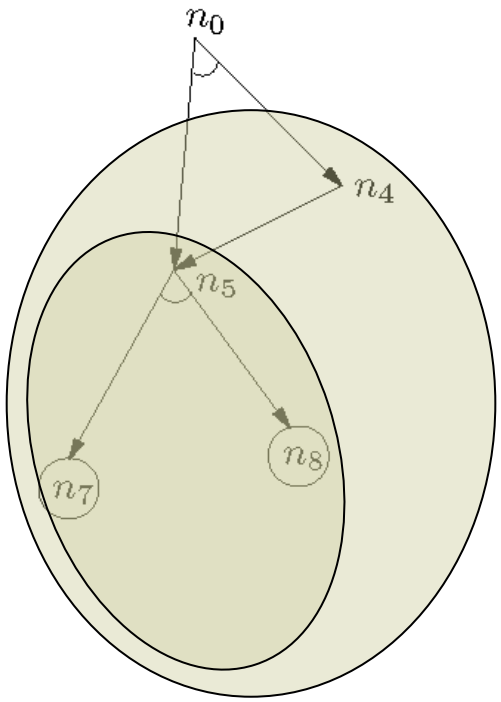
Estrategias sobre una representación por reducción

Búsqueda heurística

Ejemplo de cálculo del costo de un grafo solución:



Solución:



$K(n_0, \{n_7, n_8\}) = 2 + K(n_5, \{n_7, n_8\}) + K(n_4, \{n_7, n_8\})$

$K(n_5, \{n_7, n_8\}) = 2$

$K(n4, \{n_7, n_8\} = 1 + K(n_5, \{n_7, n_8\}) = 1 + 2 = 3$

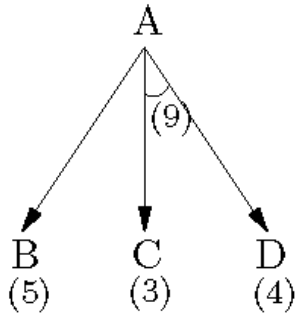
$K(n_0, \{n_7, n_8\}) = 2 + K(n_5, \{n_7, n_8\}) + K(n_4, \{n_7, n_8\}) = 2 + 3 + 2 = 7$

Estrategias sobre una representación por reducción

Búsqueda heurística

- **Características de las funciones de evaluación en grafos YO**

En general, estaremos interesados en encontrar el grafo solución que tenga costo mínimo, al que llamaremos **grafo solución óptimo**. Como es natural, dispondremos de una función heurística h que estima el costo.



El primer nodo, A, se ha expandido produciendo dos conectores, uno que conduce a B y otro a C y D.

Los números en cada nodo representan el valor de la función heurística en ese nodo, y supongamos por simplicidad que el costo de cada k-conector es k

Si simplemente miramos los nodos y elegimos para su expansión aquél con menor valor de h (como haría A*), debemos seleccionar C.

Pero usando la información que disponemos, sería mejor explorar el camino a través de B, puesto que para usar C debemos también usar D, con un costo total de 9, comparado con el costo 6 que obtendríamos por B.

Estrategias sobre una representación por reducción

Búsqueda heurística

Para buscar en un grafo YO es necesario hacer cuatro acciones en cada paso:

HACIA DELANTE:

- Atravesar el grafo empezando por el nodo inicial y siguiendo el mejor camino actual, acumulando el conjunto de nodos que van en ese camino y aún no se han expandido.
- Elegir uno de estos nodos no expandidos y expandirlo. Añadir sus sucesores al grafo y calcular h para cada uno de ellos.

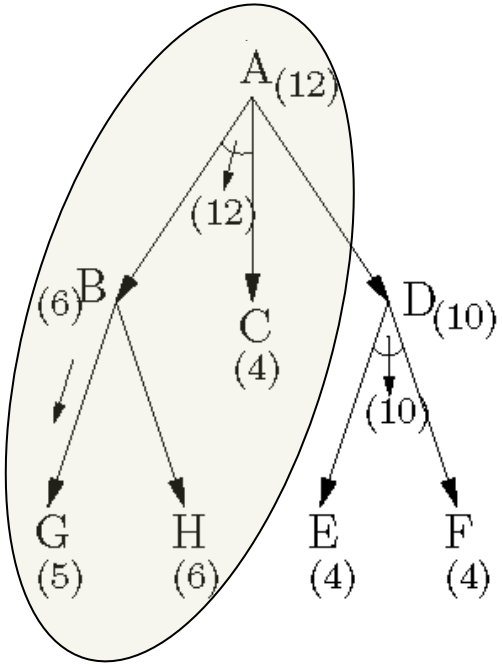
HACIA ATRÁS:

- Cambiar la h estimada del nodo recientemente expandido para reflejar la nueva información proporcionada por sus sucesores. Propagar este cambio hacia atrás a través del grafo.
- Para cada nodo que se visita mientras se va retrocediendo en el grafo, debemos decidir cuál de sus conectores es más prometedor y marcarlo como parte del mejor grafo solución parcial actual. Esto puede hacer que dicho grafo solución parcial cambie.

Estrategias sobre una representación por reducción

Búsqueda heurística

Ejemplo para mostrar esta forma de proceder en un grafo YO.



Siempre, en el nodo raíz de la estructura (problema principal) aparecerá el costo del grafo solución actual, y una marca, indicando el operador que produce este grafo solución.

Estrategias sobre una representación por reducción

Búsqueda heurística

- **Estrategia YO***

No utilizamos las dos listas, Frontera y Cerrados. Un método YO* usa una única estructura G, que representa la parte del grafo de búsqueda que se ha generado explícitamente hasta el momento (el grafo de exploración).

También se emplea un valor "Futilidad" - si el costo estimado de una posible solución se vuelve mayor que este valor entonces abandonaremos la búsqueda.

función YO*(problema) **devuelve** grafo solución

locales: G, G' grafos

G grafo vacío
G ← G+{inicio}
costo(inicio) ← h(inicio)
si inicio ∈ TERMINAL **entonces** inicio marcado resuelto
repetir hasta inicio marcado resuelto o costo(inicio)>futilidad
Construir G'⊆G con los conectores marcados
nodo ∈ frontera(G')
si no hay ningún sucesor en EXPANDIR(nodo) **entonces** costo(nodo)=futilidad

Estrategias sobre una representación por reducción

Búsqueda heurística

en otro caso ∀ sucesor ∈ EXPANDIR(nodo) **hacer**

G ← G+{sucesor}

si sucesor∈TERMINAL **entonces** sucesor marcado resuelto y costo(sucesor)=0

si sucesor∉TERMINAL y no estaba en G **entonces** costo(sucesor)=h(sucesor)

S={nodo} (S conjunto de nodos que se han marcado resuelto o cambiado su costo)

repetir hasta S vacío

actual∈S de modo que ningún descendiente en G de actual esté en S

S←S-{actual}

para cada k-conector de actual $\{n_{i1},n_{i2},\ldots,n_{ik}\}$ calcular

$$costo_i(actual)=c+costo(n_{i1})+ \ldots + costo(n_{ik})$$

costo(actual) ← min_i $costo_i$(actual)

marcar conector por el que se ha obtenido ese mínimo (borrar otra marca previa)

si todos los sucesores a través de ese conector están etiquetados como resueltos **entonces** etiquetar como resuelto actual

si actual se ha etiquetado como resuelto o se ha cambiado su costo

entonces propagar esta información hacia el principio del grafo y

S←S+{antecesores de actual}

Estrategias sobre una representación por reducción

Búsqueda heurística

Si hay un grafo solución desde el nodo inicial al conjunto de nodos terminales, y si se cumple que $h(n) \leq h^*(n)$, $\forall n$ (**heurística admisible**)



YO* termina dando un grafo solución óptimo

Una condición importante, al igual que ocurría en el espacio de estados, es la condición de monotonía de la función heurística h en una estructura YO.

Condición de monotonía de la función heurística h en **una estructura YO**:

Para cualquier k-conector, en el grafo YO implícito, dirigido desde el nodo n a los sucesores $\{n_1, \ldots, n_k\}$ se verifica que

$$h(n) \leq c(n, \{n_1, \ldots, n_k\}) + h(n_1) + \ldots + h(n_k)$$

donde $c(n, \{n_1, \ldots, n_k\})$ es el costo del k-conector.

Estrategias sobre una representación por reducción

Búsqueda heurística

A partir de la **condición de monotonía** de la función heurística h en una **estructura YO** podemos probar que:

- Si para cualquier nodo terminal el valor de h es 0, entonces la condición de monotonía de h implica la **admisibilidad de h** (es decir, $h(n) \leq h^*(n)$, $\forall n$).
- Por lo anterior, un **método YO*** guiado por una función heurística h monótona encontrará la **solución óptima**.
- Cuando se verifica la condición de monotonía de h, las revisiones de costos que se hacen en YO* sólo pueden consistir en incrementos. Por tanto, **podemos simplificar** la parte del método consistente en **la propagación hacia atrás** de la nueva información generada. La simplificación consistirá en la inclusión en S sólo de los antecesores del nodo Actual tales que Actual sea uno de sus sucesores a través de conectores marcados.

~~**si** *actual* se ha etiquetado como *resuelto* o se ha cambiado su costo **entonces** propagar esta información hacia el principio del grafo y S←S+{antecesores de *actual*}~~

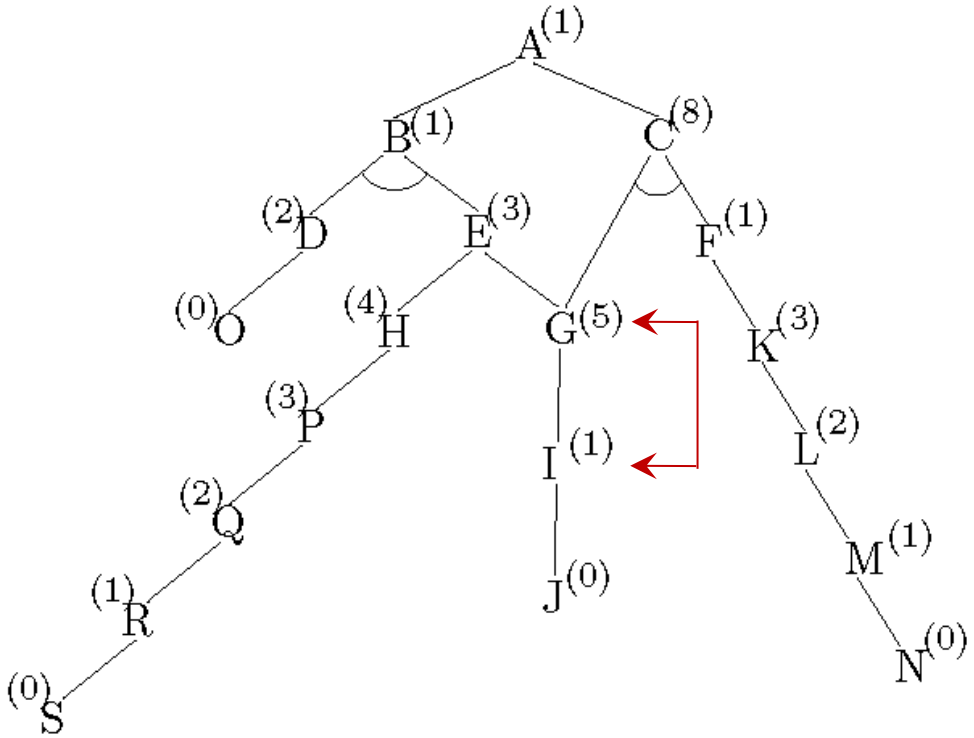


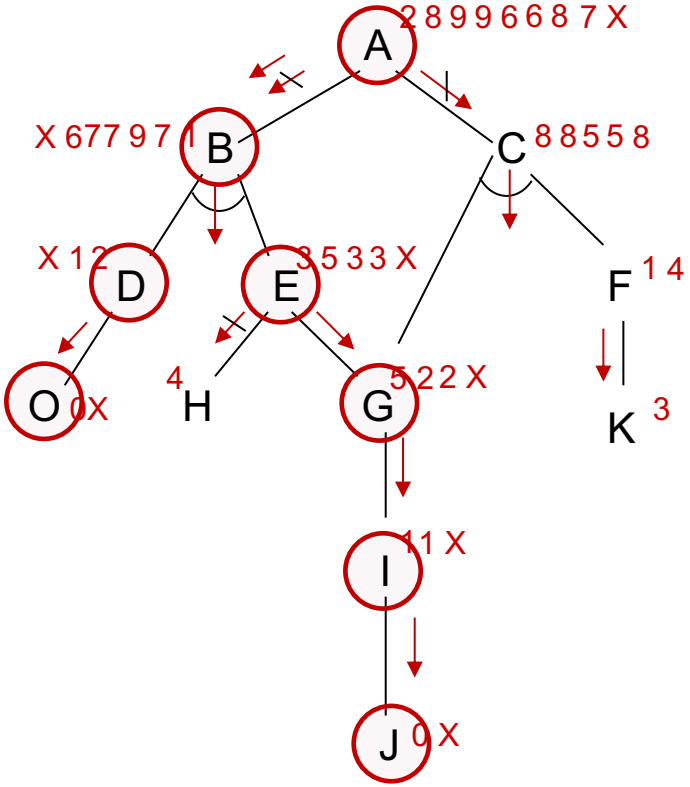
si *actual* se ha etiquetado como *resuelto* o se ha cambiado su costo **entonces** propagar esta información hacia el principio del grafo y S←S+{solo antecesores de *actual* tales que *actual* sea uno de sus sucesores a través de conectores marcados}

Estrategias sobre una representación por reducción

Búsqueda heurística

Grafo YO con heurística no monótona.





Grafo solución: costo 7

Estrategias sobre una representación por reducción

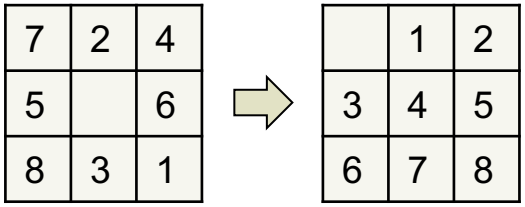
Búsqueda heurística

Tres comentarios importantes:

- Método YO* sobre grafos YO en los cuales no hay ciclos (permite poder efectuar el paso de propagar la información hacia atrás y garantizar que el método termina). Posible modificar el método YO* para trabajar con grafos YO que contengan ciclos.
- Definición de costo de un grafo solución. Supone que un sub-problema puede resolverse realmente dos (o más) veces, lo que implica contar el costo de resolverlo varias veces.
  - Problema que implica algún proceso "físico" (las soluciones que proporciona YO* son óptimas).
  - Si el proceso de resolución no es de ese tipo quizás no deberían contarse los costos más de una vez (las soluciones pueden no ser óptimas).
- Se ha supuesto que siempre que un problema (un nodo) se reduce a una conjunción de sub-problemas, todos pueden resolverse independientemente.
  - La solución de un sub-problema no tiene ningún efecto sobre la solución de otro. En muchos problemas esta suposición no está justificada (formalismo de grafos YO no adecuado) ➔ problemas mediante la planificación.

FUNCIONES HEURÍSTICAS

Analizamos algunas heurísticas para el problema 8-puzzle, para que nos den información sobre la naturaleza de las heurísticas en general.



Un caso típico del 8-puzzle. La solución tiene 26 pasos

Hay una larga historia de heurísticas para el 8-puzle, 15-puzzle; 24-puzzle, etc.

Las dos candidatas comúnmente usadas son:

- h_1 = número de piezas mal colocadas.
- h_2 = suma de las distancias de las piezas a sus posiciones en el objetivo (distancia de Manhattan).

Funciones heurísticas

EL EFECTO DE LA PRECISIÓN HEURÍSTICA EN EL RENDIMIENTO

Una manera de caracterizar la calidad de una heurística es el **factor de ramificación eficaz** b*.

Si el número total de nodos generados por un método para un problema es N y la profundidad de la solución es d, entonces el factor de ramificación eficaz b* es el factor de ramificación que un árbol uniforme de profundidad d debería tener para contener N+1 nodos.

$$N+1=1+b^*+(b^*)^2+\ldots+(b^*)^d$$

8-puzzle → coste medio de soluciones (prob. generados al azar): aprox. 22 pasos

Factor de ramificación medio: (4+3+3+3+3+2+2+2+2)/9=2,67

Búsqueda exhaustiva: $2{,}67^{22} \approx 2{,}4\times10^9$ estados

Controlando estados repetidos: 9!/2=181.440 estados

15-puzzle →

Factor de ramificación medio: (4+4+4+4+3+3+3+3+3+3+3+3+2+2+2+2)/16=3

Controlando estados repetidos: 16!/2=10,461,394,944,000 $\approx 10^{13}$ estados

Funciones heurísticas

El efecto de la precisión heurística en el rendimiento

Encontrar una buena función heurística que reduzca estos valores.

Para probar las funciones heurísticas h_1 y h_2, generamos 1200 problemas aleatorios del 8-puzzle con soluciones de longitudes de 2 a 24 (100 para cada número par).

Los resolvemos con:

- búsqueda de profundidad iterativa (BPI) y con la búsqueda A* usando tanto h_1 como h_2.

Una buena heurística debería tener b* lo más próximo a 1: ello permitiría resolver problemas complejos en tiempo razonable.

	Costo de la búsqueda			Factor de ramificación eficaz		
d	BPI	A*(h_1)	A*(h_2)	BPI	A*(h_1)	A*(h_2)
2	10	6	6	2,45	1,79	1,79
4	112	13	12	2,87	1,48	1,45
6	680	20	18	2,73	1,34	1,30
8	6.384	39	25	2,80	1,33	1,24
10	47.127	93	39	2,79	1,38	1,22
12	3.644.035	227	73	2,78	1,42	1,24
14	--	539	113	--	1,44	1,23
16	--	1.301	211	--	1,45	1,25
18	--	3.056	363	--	1,46	1,26
20	--	7.276	676	--	1,47	1,27
22	--	18.094	1.219	--	1,48	1,28
24	--	39.135	1.641	--	1,48	1,26

Funciones heurísticas

El efecto de la precisión heurística en el rendimiento

Los resultados sugieren que A*(h_2) es mejor que A*(h_1) utilización de la búsqueda de profundidad iterativa (BPI). Podemos deducir que h_2 es mejor que h_1.

Uno podría preguntarse si h_2 es siempre mejor que h_1.

La respuesta es sí.

Es fácil probar de las definiciones de las dos heurísticas, h_1 y h_2, que para cualquier nodo n, $h_2(n) \geq h_1(n)$, $\forall n$



h_2 domina a h_1

La dominancia se traslada directamente a la eficiencia:

A* usando h_2 nunca expandirá más nodos que A* usando h_1.

Funciones heurísticas

DISEÑANDO FUNCIONES HEURÍSTICAS ADMISIBLES

h_1 y h_2 son heurísticas bastante buenas para el 8-puzzle y que h_2 es mejor.

¿Cómo ha podido surgir h_2? ¿Es posible que un ordenador invente tal heurística?

- Las funciones h_1, h_2 son estimaciones de la longitud del camino restante para el n-puzzle, pero también son longitudes de caminos exactos para versiones simplificadas.

Si relajásemos las reglas del n-puzzle de modo que un ficha pudiera moverse a todas partes, en vez de solamente al cuadrado adyacente vacío, entonces h_1 daría el número exacto de pasos en la solución más corta.

Del mismo modo, si una ficha pudiera moverse a un cuadrado en cualquier dirección, hasta en un cuadrado ocupado, entonces h_2 daría el número exacto de pasos en la solución más corta.

Funciones heurísticas

Diseñando funciones heurísticas admisibles

Un problema con menos restricciones en las acciones se le llama **problema relajado**. El costo de una solución óptima en un problema relajado es una heurística admisible para el problema original.

La heurística es admisible porque la solución óptima en el problema original es, por definición, también una solución en el problema relajado y por lo tanto debe ser al menos tan cara como la solución óptima en el problema relajado.

Problema escrito en lenguaje formal ➔ construir problemas relajados automátic.

8-puzzle:

- Una ficha puede moverse del cuadrado A al cuadrado B si
  - A es horizontalmente o verticalmente adyacente a B y B es la vacía.

Podemos generar varios problemas relajados quitando una o ambos condiciones:

(a) Una ficha puede moverse del cuadrado A al cuadrado B si A es adyacente a B.
(b) Una ficha puede moverse del cuadrado A al cuadrado B.

con (a) obtenemos h_2 con (b) obtenemos h_1

Si definición de otra manera → nuevas heurísticas.

Funciones heurísticas

Diseñando funciones heurísticas admisibles

Un problema con la generación de nuevas funciones heurísticas es que a menudo se falla en conseguir una heurística "claramente mejor".

Si tenemos disponible un conjunto de heurísticas admisibles $h_1, \ldots, h_m$ para un problema, y ninguna de ellas domina a las demás, ¿qué deberíamos elegir?



No tenemos por que hacer una opción



$$h(n) = \max \{ h_1(n), \ldots, h_m(n) \}, \forall n$$

Como las heurísticas componentes son admisibles, h es admisible (es también fácil demostrar que h es consistente).

Además, h domina a todas sus heurísticas componentes.